Computability Notes

1 Definitions

A problem is **computable** if there is some algorithm that solves it in finite time.

Given a decision problem and a program M that takes an input x, we define

$$\mathcal{L}_M = \{\langle x \rangle \mid M(x) \text{ terminates in finite time and returns } \textit{true}\}.$$

Then:

- $\mathcal{L}_M$ is **recognized** by M if $M(x)$ returns *true* iff $x \in \mathcal{L}_M$. If $x \notin \mathcal{L}_M$, then $M(x)$ may either (1) return *false* or (2) run indefinitely.
- $\mathcal{L}_M$ is **decided** by M if $M(x)$ returns *true* iff $x \in \mathcal{L}_M$ and returns *false* iff $x \notin \mathcal{L}_M$.

Notice that Decidability $\subset$ Recognizability (i.e. Decidable $\Rightarrow$ Recognizable, and Not Recognizable $\Rightarrow$ Not Decidable).

2 Known Problems

2.1 Halting Problem

Given a program M that takes an input x, let

$$\mathcal{L}_{\text{HALT}} = \{\langle M, x \rangle \mid M(x) \text{ terminates in finite time}\}.$$

Properties:

- **Recognizable**: A program $H(M, x)$ can simply call $M(x)$ and return *true* if $M(x)$ terminates.
- **Not decidable**: Proved via diagonalization

2.2 Co-Halting Problem

(The complement of the Halting Problem). Given a program M that takes an input x, let

$$\mathcal{L}_{\text{CO-HALT}} = \{\langle M, x \rangle \mid M(x) \text{ never terminates (i.e. runs forever)}\}.$$

Properties:

- **Not recognizable**: Proved via diagonalization

2.3 Acceptance Problem

Given a program M that takes an input x, let

$$\mathcal{L}_{\text{ACCEPT}} = \{\langle M, x \rangle \mid M(x) \text{ returns } \textit{true} \text{ in finite time}\}.$$

Properties:

- **Recognizable**: A program $H(M, x)$ can simply call $M(x)$ and return *true* if $M(x)$ returns *true*.
- **Not decidable**: Proved via reduction: $\mathcal{L}_{\text{HALT}} \leq \mathcal{L}_{\text{ACCEPT}}$. Define M' with input x as follows: if $M(x)$ terminates, then return *true*. Then, `acceptanceChecker` and subsequently `haltChecker` both return *true* iff $M(x)$ terminates.

2.4 Totality Problem

Given a program M, let

$$\mathcal{L}_{\text{TOTAL}} = \{\langle M \rangle \mid M \text{ terminates on all inputs } x\}.$$

Properties:

- **Not recognizable:** Proved via reduction: $\mathcal{L}_{\text{CO-HALT}} \leq \mathcal{L}_{\text{TOTAL}}$. Define M' with input y as follows: simulate $M(x)$ for $|y|$ iterations. If M terminates within $|y|$ iterations, then loop forever. Otherwise, terminate M'. Then, if M terminates after t iterations, then M' will not terminate on finitely many inputs $|y| \geq t$. If M never terminates, then M' will also terminate.
- **Not decidable:** Proved via reduction: $\mathcal{L}_{\text{HALT}} \leq \mathcal{L}_{\text{TOTAL}}$. Define M' as follows: Have M' ignore its input, and simply simulate $M(x)$. Then, as long as $M(x)$ terminates, both `totalityChecker` and `haltChecker` will return *true*.

2.5 Busy Beaver Problem

Given an integer n, let

$$B(n) = \text{ maximum number of steps any program of size } n \text{ can run before halting.}$$

Properties:

- **Not computable**: Proved via reduction: $\mathcal{L}_{\text{HALT}} \leq$ computing $B(n)$.

3 Proofs

3.1 Recognizable

1. **Define a recognizer.** Define a program H that takes the same input(s) as the original problem.
2. **Describe its behavior.** Describe the behavior of H that would make it return *true*, which typically includes either (1) a simulation if the original problem takes in inputs of program M and variable x, or (2) some condition that the input variable x must satisfy.
3. **Bidirectional proof**. Prove that if the input(s) to the original problem is in $\mathcal{L}$, then H returns *true*. Also, prove that if H returns *true*, then the input(s) to H is in $\mathcal{L}$.

3.2 Not Recognizable

1. **Define the language for the original problem.** Define $\mathcal{L}$, the set of all inputs such that some property (as defined by the original problem) holds.
2. **Define two recognizers.** Suppose that we have some program `LRecognizer` that correctly returns *true* iff the input(s) is in $\mathcal{L}$. Since the Co-Halting Problem is not recognizable, our goal is to show that $\mathcal{L}_{\text{CO-HALT}} \leq \mathcal{L}$. To do this, we define another program `cohaltRecognizer` that takes inputs M and x.
3. **Define a modified program.** Define a program M' with the following behavior: first, have M' initially behave in a way that satisfies the condition(s) of the original problem. Next, simulate $M(x)$. If $M(x)$ halts, then force M' to do something so that it no longer meets the condition(s). Otherwise, do nothing. Finally, supply M' as the input to `LRecognizer`, call `LRecognizer` in `cohalfRecognizer`, and return its output.
4. **Proof by contradiction.** Prove the correctness of `cohaltRecognizer`, i.e. if $M(x)$ does not terminate, then we forced M' to meet the conditions of the original problem, so `LRecognizer` and subsequently `cohaltRecognizer` return *true*. If $M(x)$ terminates, show that M' no longer meets the conditions, so both recognizers return *false*. We observe that `cohaltRecognizer` correctly recognizes $\mathcal{L}_{\text{CO-HALT}}$ under the assumption that `LRecognizer` recognizes the original problem correctly. However, as the Co-Halting Problem is unrecognizable, it must be the case that `LRecognizer` is not a valid recognizer.

3.3 Decidable

1. **Define a decider.** Define a program H that takes the same input(s) as the original problem.
2. **Describe its behavior.** Describe the behavior of H that would make it return *true*, as well as its behavior that would make it return *false*. For instance, you may have to construct a helper method that H calls.
3. **Prove that H terminates.** Show that for any input(s), H terminates in finite time. (This is because by definition of decidability, the decider must always return some value. Otherwise, $\mathcal{L}$ is only recognizable.) For instance, you can bound the total number of iterations or recursive calls, show that the search space is finite, etc.
4. **Bidirectional proof.** Consider both cases: show that if the input(s) to the original problem is in $\mathcal{L}$, then H returns *true*, and if the input(s) is not in $\mathcal{L}$, then H returns *false*.

3.4 Not Decidable

1. **Define the language for the original problem.** Define $\mathcal{L}$, the set of all inputs such that some property (as defined by the original problem) holds.
2. **Define two deciders.** Suppose that we have some program `LChecker` that correctly decides whether the input(s) is in $\mathcal{L}$. Since the Halting Problem is not decidable, our goal is to show that $\mathcal{L}_{\text{HALT}} \leq \mathcal{L}$. To do this, we define another program `haltChecker` that takes inputs M and x.
3. **Define a modified program.** Define a program M' with the following behavior: first, simulate $M(x)$. If $M(x)$ halts, then do something such that the condition(s) for the original problem is satisfied, and return *true*. Finally, supply M' as the input to `LChecker`, call `LChecker` in `haltChecker`, and return its output.
4. **Proof by contradiction.** Prove the correctness of `haltChecker`, i.e. if $M(x)$ terminates, then `haltChecker` returns *true*, and returns *false* otherwise. We observe that `haltChecker` correctly decides $\mathcal{L}_{\text{HALT}}$ under the assumption that `LChecker` decides the original problem correctly. However, as the Halting Problem is undecidable, it must be the case that `LChecker` is not a valid decider.